

μEZ[®] v2.15 Software User's Guide



FDI *Future Designs, Inc.*
Your Development Partner
996 A Cleaner Way, Huntsville, AL 35805

Copyright ©2025, Future Designs, Inc., All Rights Reserved

Information in this document is provided solely to enable the use of Future Designs products. FDI assumes no liability whatsoever, including infringement of any patent or copyright. FDI reserves the right to make changes to these specifications at any time, without notice. No part of this document may be reproduced or transmitted in any form or by any means, electronic or mechanical, for any purpose, without the express written permission of Future Designs, Inc. 996 A Cleaner Way, Huntsville, AL 35805

For more information on FDI or our products please visit www.teamfdi.com.

NOTE: The inclusion of vendor software products in this kit does not imply an endorsement of the product by Future Designs, Inc.

© 2025 Future Designs, Inc. All rights reserved.

uEZ is a registered trademark of Future Designs, Inc.

Microsoft, MS-DOS, Windows, Windows XP, Microsoft Word are registered trademarks of Microsoft Corporation.

Other brand names are trademarks or registered trademarks of their respective owners.

FDI PN: MA00016

Revision: 2.15, 07/29/2025 2:00:00 PM

Printed in the United States of America

Revision History

Who	Date	Details
L. Shields	09/10/2008	Initial version.
L. Shields	10/08/2008	UEZSystemInit() added. UEZTaskDelay() changed to only delay current task.
P. Quirk	10/10/2008	Updated introductory μ EZ section, renumbered sections to segment OSAL from Subsystem API, recommendation in section 3 to remove UEZ prefix on functions, parameters, and return codes.
L. Shields	10/15/2008	Added diagram. Updated documents.
L. Shields	10/28/2008	Added extensive information about existing subsystem device drivers and HAL drivers.
L. Shields	10/29/2008	Added serial/stream docs. Added HAL and device driver struct definitions for each system.
L. Shields	11/18/2008	Added example widget drivers and library in appendix.
L. Shields	11/19/2008	Finally replaced all uFlex with μ EZ
L. Shields	11/26/2008	Added file I/O routines (UEZFile...)
L. Shields	12/03/2008	Added lwIP TCP/IP functions.
L. Shields	12/05/2008	Added I2C routines
L. Shields	12/05/2008	Added SWIM library routines.
L. Shields	01/12/2009	Added USB Device HAL driver and USB Device Driver information. SetPalette routines added to LCD drivers. Touch sensitivity routines added to touch screen drivers and library.
L. Shields	02/03/2009	Added FileStart/Stop/Next routines for doing file enumeration in a given directory.
L. Shields	02/03/2009	Added RTC Device, HAL, and μ EZ API. Added Mass Storage device driver. Added SPI new functions both in device and HAL layers. Added Backlight device driver. Updated LCD device driver Configure() function. Added USB Host device driver API.
L. Shields	02/18/2009	Added Accelerometer, Button Bank, EEPROM, and Temperature device driver. Added PWM HAL driver.
L. Shields	03/24/2009	Change UEZTSClose() to properly remove TS queue. Added boot process section. Added platform and processor creation details.
L. Shields	06/08/2009	Added new RTC driver functions Validate and SetClockOutHz. Modified Platform initialization documented (uEZPlatformStartup).
L. Shields	07/22/2009	Updated for μ EZ v1.03. Added more details on platform startup. Formatting changes.
L. Shields	09/02/2009	Modified introduction for more info about FDI.
L. Shields	09/02/2009	Added Serial and Stream Flush commands
L. Shields	02/20/2010	Added UEZADC, UEZCharDisplay, UEZKeypad UEZLEDBank, UEZTemperature, UEZToneGenerator routines. Added UEZFileGetLength()
L. Shields	04/01/2010	Added notes about SPI and SSP data direction of iDataIn and iDataOut.
L. Shields	04/02/2010	Added HAL_DAC. Added HAL_BatteryRAM.
L. Shields	05/24/2010	Added Flash device driver and system functions.
C. Stocker	05/13/2011	Added Audio Codec, watchdog, DAC EINT, and I2S device drivers. Added uEZWAVFile, uEZWatchDog, uEZDAC, and uZEINT. Added I2S HAL driver.
C. Stocker	07/12/2011	Added uEZ Network API and uEZ INI Library.
L. Shields	07/14/2011	Additional text in uEZ Network API and uEZ INI Library.
L. Shields	09/01/2011	Added uEZHID routines.
L. Shields	10/18/2011	Added SetMatchCallback to HAL Timer and HAL PWM. Added SetMatchCallback to Device PWM. Put in more page breaks between the sections to help break up the sections.
B. Fleming	07/13/2012	Updated for μ EZ 2.01
B. Fleming	11/28/2012	Updated for μ EZ 2.03; Remove Subsystem Device Drivers
B. Fleming	09/15/2014	Updated for μ EZ 2.06; Remove LPC2478 references as it is no longer supported, added RX63N, LPC4088 to support list, fix some sentence structure mistakes.
C. Cox	12/27/2022	Updated the time, copyright, and date information. Long overdue.
C. Cox	07/11/2023	Updates for μ EZ 2.13
B. Fleming	07/29/2025	Updates for μ EZ 2.15

Table of Contents

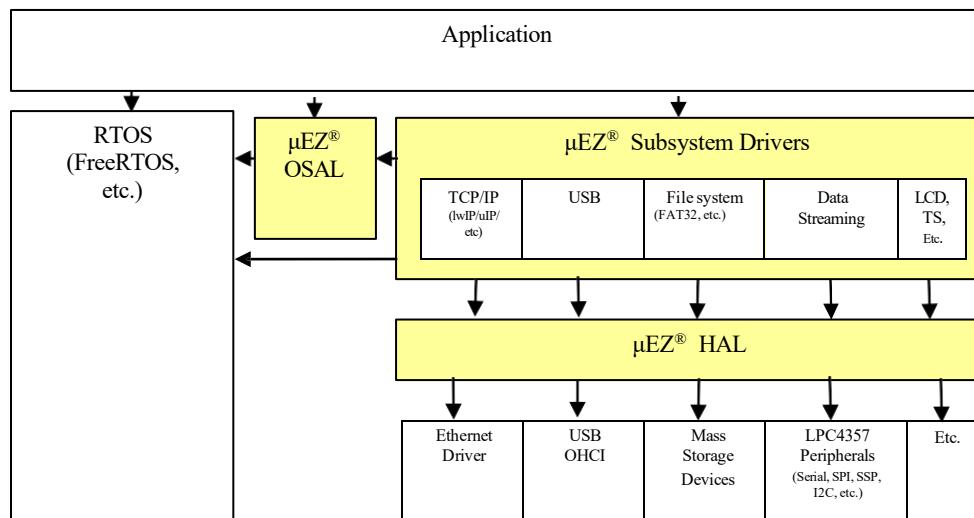
1.0	μEZ.....	5
1.1	Basic Types.....	7
2.0	Subsystem driver API.....	8
3.0	HAL Layer.....	9
3.1	HAL Registration.....	11
3.1.1.	HALInterfaceRegister.....	11
3.1.2.	HALInterfaceGetWorkspace	11
3.1.3.	HALInterfaceFind	12
4.0	Libraries.....	13
4.1	Light Weight IP (lwIP) TCP/IP Stack.....	13
4.2	Simple Windowing Interface Manger (SWIM)	14
4.3	USB Device Generic Bulk Library.....	14
4.3.1.	Concepts and Overview	14
4.3.2.	On the PC Side.....	14
4.3.3.	Usage	14
4.3.3.1.	GenericBulkInitialize.....	15
4.3.3.2.	GenericBulkPut.....	15
4.3.3.3.	GenericBulkGet	15
4.3.4.	More Information about libusb library	15
5.0	The uEZ Boot Process.....	16
6.0	Setting up a New Processor.....	17
6.1	Overview.....	17
6.2	Processor Specific RTOS.....	17
6.3	Adding uEZ Processor Device Driver Files.....	17
6.4	Processor Initialization	18
6.4.1.	Step 1: Register the HAL Driver Interface	18
6.4.2.	Step 2: Configure the HAL Driver.....	18
6.4.3.	Miscellaneous Initialization.....	19
6.5	Interrupt Table.....	19
6.6	Testing a New Processor	19
7.0	Setting up a New Platform.....	20
7.1	Overview.....	20
7.2	Platform Files	20
7.3	Platform Configuration.....	20
7.4	Platform Initialization.....	21
7.4.1.	Device Driver Registration and Initialization.....	21
7.4.1.1.	Step 1: Register the Device Driver Interface	21
7.4.1.2.	Step 2: Configure the Device Driver	21
7.4.2.	Additional Startup Steps	22
7.4.3.	Platform Startup Task.....	22
8.0	Appendix: Example Widget Driver Implementation.....	23
8.1	HAL Driver.....	23
8.2	Device Driver	25
8.3	Library.....	28
8.4	Summary.....	30

1.0 μ EZ

μ EZ is an open source, middleware platform so there is no cost to the user. Customers can directly integrate μ EZ into their embedded application for free or can contract with Future Designs to provide affordable integration services that are customized to your hardware and software requirements. Future Designs integration services are full turnkey and can cost as little as \$10K and may require only 4 weeks of schedule time to complete and test. The goal of the μ EZ platform is to provide underlying RTOS and processor abstraction enabling the application programmer to focus on the value-added features of their product. μ EZ is an *optional* platform that enhances portability of application code to multiple embedded MCU platforms with high reusability.

μ EZ has three primary components which are highlighted in the diagram below:

- Operating System Abstraction Layer (OSAL)
- Subsystem drivers
- Hardware Access Layer (HAL)



The OSAL is the primary component of μ EZ and provides applications access to the following features in an OS-independent fashion:

- Pre-emptive multitasking core
- Stack overflow detection options
- Unlimited number of tasks
- Queues
- Semaphores (binary, counting, and mutex)

The Subsystem drivers utilize the abstracted RTOS OSAL functions to provide protected access to the processor peripherals. The Subsystem drivers also support direct usage of FreeRTOS when the μ EZ platform is not utilized. The HAL functions provide single-threaded unprotected access to the processor peripherals. Customers can use the μ EZ HAL routines provided by Future Designs or can write their own. When written correctly, the HAL routines provide for RTOS and μ EZ independence.

μ EZ 2.15 currently supports the following processor families:

- NXP LPC1788
- NXP LPC4088
- NXP LPC4357
- More processors to come ...

μ EZ currently supports the following RTOS families:

- FreeRTOS

1.1 Basic Types

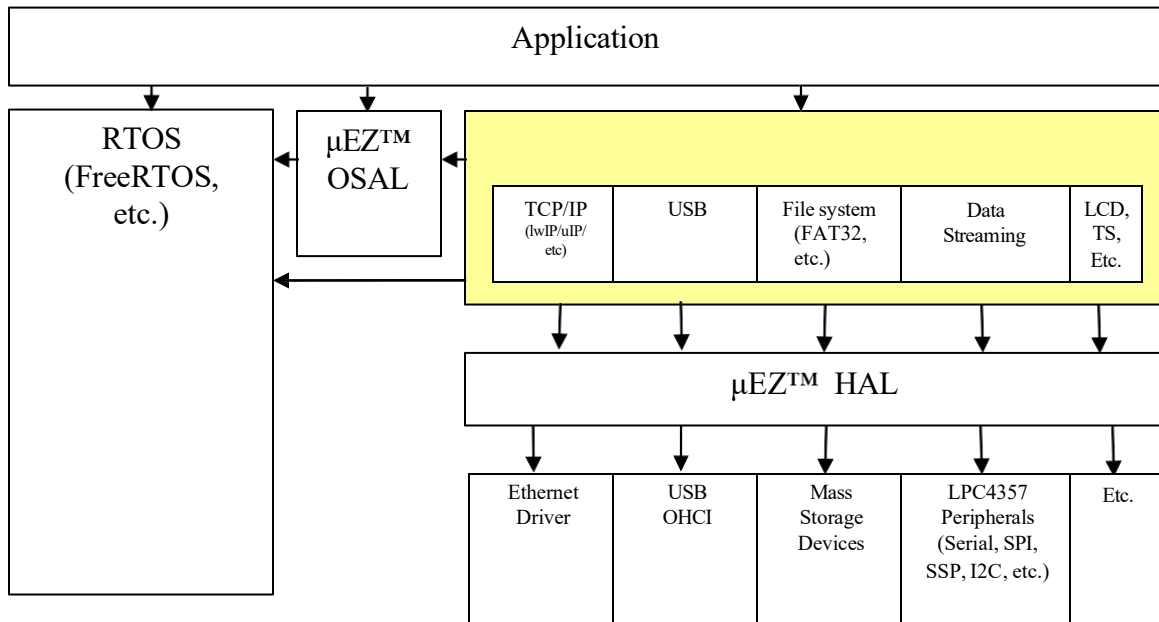
μEZ uses the following data types for all of its routines.

Name	Type	Notes
TUInt32	32-bit unsigned integer	
TInt32	32-bit signed integer	
TUInt16	16-bit unsigned integer	
TInt16	16-bit signed integer	
TUInt8	8-bit unsigned integer	
char	8-bit signed integer	
TBool	Boolean value	Use ETrue, or EFalse for values.
T_uezHandle	32-bit handle	Unique identifier for many system wide resources.
T_uezError	Enum Type	uEZ-related return codes.

2.0 Subsystem driver API

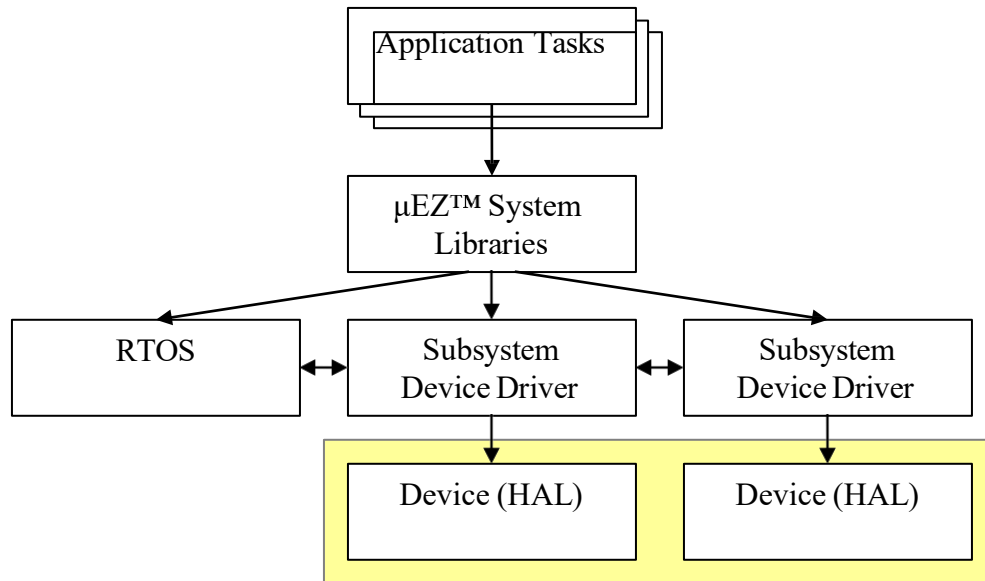
The μ EZ Subsystem drivers wrap around existing functions and provide a common API to commonly used devices. Although not required, the API provides a good mechanism for code re-use.

See the topic section in <https://www.teamfdi.com/uezgui-doxygen/> for more information on the Subsystem driver API.



3.0 HAL Layer

The HAL Layer is the layer of routines that tie directly to the hardware. Routines at this layer do not know about the upper layers of the application, libraries, or even RTOS. Semaphores, queues, and other RTOS constructs are handled by the Subsystem Device Driver layer. At the HAL layer, each function assumes it is the only code running on select section of the hardware.



Families of HAL interface types are defined in the directory “Include/HAL”. For example, a project may be using a serial UART on the processor and a serial UART on the base board. Both of these are serial UARTs and their HAL drivers will use the HAL_Serial interface (see “Include/HAL/Serial.h”).

Each HAL device must provide a HAL interface. The HAL interface is a structure registered at power on and starts with a T_halInterface substructure. T_halInterface is as follows:

```
typedef struct {
    const char *iName;
    TUInt16 iVersion;    // Version is 0x103 for version 1.03
    T_uezError (*iInitializeWorkspace)(void *aWorkspace);
    TUInt32 iWorkspaceSize;
} T_halInterface;
```

Each field starts with a pointer to a unique name. Names are given in the following format “<device type>:<company>:<hardware>”. For example “Touchscreen:TI:TSC2046” means it is a touch screen device using Texas Instrument’s TSC2046 integrated circuit.

Next comes the 16-bit version number. The upper byte is the major version and provides compatibility information – the number must match to be compatible. The lower byte is the minor version. The minor version must be equal or greater to the required minor version to be compatible. For example, version 0x0203 is needed for a project. If the HAL driver is 0x0300 or 0x0100, the major version is different and definitely not compatible. A HAL driver of version 0x0200 is also not compatible because it has a lower minor version. However, a HAL driver of version 0x0204 is compatible even though it may have a few other extra features.

The fields `iInitializeWorkspace` and `iWorkspaceSize` are used when the HAL driver is registered. A workspace is allocated on the heap and calls `iInitializeWorkspace` to setup the driver.

The rest of the HAL interface structure is typically a list of function pointers that expose the features of the HAL device (although other fields can be provided as required by the standard interface).

3.1 HAL Registration

When a processor or platform initializes, the HAL interface is registered to the uEZ system and a workspace instance created. By registering, other devices can find the system using a string name that matches the one used at registration. By registering unique devices to generic devices names (e.g., use “SPI0” for “SPI:NXP:LPC1788:SPI0”), the rest of the system can quickly map to the hardware without knowledge of the particular hardware type being used.

3.1.1. **HALInterfaceRegister**

Description:

Registers a new HAL interface and sets up a workspace (instance) for it.

Parameters:

const char * const aName – Pointer to name to register the device as.

T_halInterface *aInterface – Pointer to HAL interface.

T_uezInterface *aInterfaceHandle – Pointer to newly created interface handle.

T_halWorkspace **aWorkspace – Returns a pointer to the newly created and initialized workspace.

Results:

T_uezError – If successful, returns UEZ_ERROR_NONE. If memory for the workspace cannot be allocated, returns UEZ_ERROR_OUT_OF_MEMORY. If there is no more space in the HAL registration table, returns UEZ_ERROR_OUT_OF_HANDLES. If initialization of the workspace fails, an error specific to that HAL may also be returned.

3.1.2. **HALInterfaceGetWorkspace**

Description:

Using a HAL’s interface handle, returns a pointer to the HAL instance’s workspace.

Parameters:

T_uezInterface *aInterfaceHandle – Pointer to interface handle.

T_halWorkspace **aWorkspace – Returns a pointer to the workspace.

Results:

T_uezError – If successful, returns UEZ_ERROR_NONE. Otherwise, returns an error code.

3.1.3. **HALInterfaceFind**

Description:

Searches for a HAL instance by name.

Parameters:

const char * const aName – Name to search for.

T_halWorkspace **aWorkspace – Returns a pointer to the workspace if found.

Results:

T_uezError – If successful, returns UEZ_ERROR_NONE. Otherwise, returns UEZ_ERROR_NOT_FOUND.

4.0 Libraries

4.1 Light Weight IP (lwIP) TCP/IP Stack

lwIP is a small independent implementation of the TCP/IP protocol suite that has been developed by Adam Dunkels at the Computer and Networks Architectures (CNA) lab at the Swedish Institute of Computer Science (SICS).

The focus of the lwIP TCP/IP implementation is to reduce resource usage while still having a full scale TCP. This making lwIP suitable for use in embedded systems with tens of kilobytes of free RAM and room for around 40 kilobytes of code ROM.

In uEZ 2.15, we updated uEZ to use lwIP v2.2.0.

LwIP is well documented on the web. One of those locations is <http://www.nongnu.org/lwip/>

Note: Each new RTOS task that gets created to call into lwIP functions should first call `lwip_socket_thread_init();`.

4.2 Simple Windowing Interface Manger (SWIM)

The SWIM library is a small window manager library created by NXP for embedded applications that need a lightweight graphics library for drawing pixels, lines, boxes, text, and images on LCD devices.

SWIM has been around a long time (15+ years) and the user's manual is from 2009.

It can be found here:

https://community.nxp.com/pwmxy87654/attachments/pwmxy87654/lpc/1038/1/EEOL_2009JUL10_EMS_CTRLD_AN_01.pdf

4.3 USB Device Generic Bulk Library

The Generic Bulk library is an example library that shows how to enable the target to become a USB Device that uses the 'usbllib' PC and Linux library to communicate over USB bulk endpoints.

4.3.1. Concepts and Overview

The Generic Bulk library was created with the idea that a developer may wish to communicate between the target (device) and the PC (host) similar to a serial port. Data is streamed from the host to the target and back. To accomplish this, the uEZ GenericBulk library uses a GenericBulkGet and GenericBulkPut command with queues to block until data is ready or sent. While the get and put routines are blocking, the GenericBulkMonitor is monitoring USB communications and doing the work of taking data off the send queue, sending it out, and then stuffing responses back on the receive queue. This is all done by making continuous calls to ProcessEndpoints on the USB Device Driver.

The routines GenericBulkBulkIn() and GenericBulkBulkOut() handle the data passed between the USB Device Driver and the queues. They read one block at a time and pass it on.

Bulk endpoints are used to send and receive data. Because bulk data is always 64 bytes (or larger), the data that is sent and received uses a one-byte header that declares the actual length of the data used in the block. In this way, smaller blocks of data can be passed back and forth.

4.3.2. On the PC Side

Included with the source code is an example Visual C++ 6.0 project called Utilities\USB\libusb_bulk_example\libusb_example. This example starts up, sends "Hello" to the device, and then monitors all data received to outputs the data to the console screen.

4.3.3. Usage

To use the GenericBulk library, just call GenericBulkInitialize() providing a structure to any callbacks you may want (in specific, if you want a callback routine each time the output queue becomes empty). The hardware is initialized and monitor task is created to handle data transfers in and out of the queues. Use GenericBulkPut and GenericBulkGet

to send and receive data. Library Routines

4.3.3.1. **GenericBulkInitialize**

Description:

Initialize the GenericBulk system. The hardware is initialized and the queues created. A monitoring task called “USBDevice” is created. Once completed, the target can be plugged into a USB Host port.

NOTE: If using both a USB Device and USB Host configuration, call GenericBulkInitialize() after the USB Host has been initialized.

Parameters:

T_GenBulkCallbacks *aCallbacks – Pointer to callback routines.
Currently only a single callback when the output queue (from device to host) becomes empty.

Results:

T_uezError – UEZ_ERROR_NONE if successful, else error code.

4.3.3.2. **GenericBulkPut**

Description:

Put character into the output queue (from device to host) or timeout trying.

Parameters:

TUInt8 c – Character to output
TUInt32 aTimeout – Length of time in milliseconds to wait before timing out.

Results:

TBool – Returns ETrue if successful, else EFalse.

4.3.3.3. **GenericBulkGet**

Description:

Retrieve a character from the incoming queue (from host to device).

Parameters:

TUInt32 aTimeout – Length of time in milliseconds to wait before timing out.

Results:

TInt32 – Character received (0 – 0xFF) or -1 if failed.

4.3.4. **More Information about libusb library**

The Windows version of the libusb library is available at <https://libusb-win32.sourceforge.net/>

The Linux version of the libusb library is available at <https://libusb.wiki.sourceforge.net/>

Check both sites for up to date versions.

5.0 The uEZ Boot Process

The uEZ system goes through the following steps before application code is executed. If a uEZ bootloader is used it will follow these same steps, then typically the application will repeat them after a reset and vector table change.

- 1) Execution starts in the startup.s or bootstrap code where the memory is initialized and the stacks are configured.
- 2) UEZBSPStartup() is called (see file Source/uEZSystem/uEZBSPInit.c)
 - a. UEZBSP_InterruptsReset is called to re-initialize the interrupts in the system.
 - b. UEZSystemInit() is an internal call to uEZ to setup the registries and the core uEZ features.
 - c. uEZProcessorServicesInit is called to set up all the hardware HAL device drivers provided by the processor.
 - d. Next, uEZPlatformInit() is called to setup all the device drivers (typically non-HAL based) provided by all devices external to the processor.
 - e. UEZTaskCreate then creates a startup task – uEZPlatformStartup.
 - i. uEZPlatformStartup is a task that initializes various peripheral functionality, such as temp, EEPROM, and RTC, then creates the MainTask, but it does not start executing right away.
 - j. MainTask performs any final setup and miscellaneous tasks.
 - f. UEZSystemMainLoop is then run, which kicks off the MainTask.
NOTE: MainTask never returns.
- 3) MainTask() is now executing.

6.0 Setting up a New Processor

6.1 Overview

uEZ was designed to allow portability between multiple processors with little to no changes to the code. Porting uEZ to new processors requires the addition of two primary steps: installing a compatible RTOS and adding support files for the processor.

6.2 Processor Specific RTOS

A RTOS with the new processor feature must be added first. If the RTOS does not support the new processor, then the RTOS will have to be modified first. Follow the instructions for porting the RTOS as provided by the maker of the RTOS. Place the resulting build in the Source/RTOS directory. If the code is to be submitted back to the uEZ sourceforge.net page, the RTOS should be modified by adding the new processor supported, not just replacement.

6.3 Adding uEZ Processor Device Driver Files

In effect, adding a new processor is simply the addition of a collection of HAL device drivers. All processor device drivers are put in one directory under the manufacturer and processor name as follows:

```
/Source/Processor/<manufacturer>/<processor name>
```

For example,

```
/Source/Processor/NXP/LPC1788
```

specifies the directory for all the LPC1788 files.

Each processor specific HAL driver should be prefixed with the processor name with an underbar (e.g. LPC1788_) to avoid filename conflicts.

All processor types are defined in the file Include/Config.h. Add the new processor to the list of processor types in MANUFACTURER_PROCESSOR format and in all caps (e.g., NXP_LPC1788) and use the next sequential number. Change the #define UEZ_PROCESSOR to be the newly defined MANUFACTURER_PROCESSOR name.

To ensure that the correct device drivers compile, in each processor HAL device driver .c and/or .s file include the following header and footer:

Header:

```
#include "Config.h"
#if (UEZ_PROCESSOR==MANUFACTURER_PROCESSOR)
```

Footer:

```
#endif // (UEZ_PROCESSOR==MANUFACTURER_PROCESSOR)
```

Finally, add all these new files to the project make file. For example, if using Rowley Crosswork's CrossStudio, to the Project Explore, create a new folder under processors with a matching name, right click on the new folder, and select "Add Existing File...". Select all the files needed by the processor and click **Open**.

6.4 Processor Initialization

Create a final file called <Processor>_HALInit.c (e.g., LPC1788_HALInit.c) copying an existing one from another project if necessary and add it the project. This file will also need the headers and footers as previously described.

This file only contains one routine called uEZProcessorServicesInit() and is called early in the boot sequence to register and configure all the hardware devices specific to this processor.

In general, each peripheral follows these two basic steps.

6.4.1. Step 1: Register the HAL Driver Interface

Call HALInterfaceRegister() with the name and HAL interface. If the hardware device is needed by another HAL device, the return pointer to the workspace can be optionally returned. For example:

```
HALInterfaceRegister(  
    "GPIO0",  
    (T_halInterface *)&GPIO_LPC1788_Port0_Interface,  
    0,  
    &p_port);
```

In this case, the HAL driver is defined by GPIO_LPC1788_Port0_Interface, will be defined in the registry as “GPIO0”, and returns a pointer to the T_halWorkspace in p_port.

The HAL driver interfaces are defined inside each of the individual HAL drivers. For each interface, add the following line at the top:

```
extern const HAL_GPIOPort GPIO_LPC1788_Port0_Interface;
```

In this case, a HAL_GPIOPort is defined, but the type will change with the different class types.

6.4.2. Step 2: Configure the HAL Driver

Next call an appropriate configure routine if necessary. Each device driver will have a different routine for configuring, and some will have none. HAL drivers may depend on each other and the returned workspace pointer will be passed in to the next configure routine. In this way, they can be linked together. When configuration is required, a specific routine know only by that driver and with the word Configure in it will be used. To continue the GPIO0 port initialization example, the following is used.

```
((HAL_GPIOPort *) (p_port->iInterface))->Configure(  
    p_port,  
    0xFFFFFFFF);
```

In this case, the GPIO driver wants to know which of the pins to allow in the port definition and a value of 0xFFFFFFFF signifies all 32 port pins are available on this processor version.

6.4.3. Miscellaneous Initialization

Any additional configuration needed may also go in the `uEZProcessorServicesInit()` routine, but it is recommended that this be kept to a minimum. The `uEZProcessorServicesInit()`'s primary purpose is to register devices, not go through specific processor initialization. These steps should be handled by the bootstrap code.

6.5 Interrupt Table

A core feature of all processors is the interrupt table. Interrupts are referenced by 'channels' and priorities. Although possibly already provided by the RTOS, the following routines must be provided to be compatible with the uEZ system (see `Include/HAL/Interrupt.h` for complete function prototypes).

```
InterruptsReset  
InterruptRegister  
InterruptUnregister  
InterruptEnable  
InterruptDisable  
InterruptDisableAllRegistered  
InterruptEnableAllRegistered
```

6.6 Testing a New Processor

It is recommended that a new processor be tested with a known good hardware configuration (bootstrap works, memory initialized, and ability to program code into part) along with a good debugger to step through the code. Use the debugger to step through the `uEZProcessorServicesInit()` and make sure the processor is setup in the proper order. Unfortunately, at this point, each device must be tested one by one with an appropriate test program and is target dependent. Usually it is best to test one hardware component at a time, possibly leaving out the others in `uEZProcessorServicesInit()` until confirmed.

7.0 Setting up a New Platform

7.1 Overview

A uEZ platform is viewed as a complete hardware solution using a specific processor and a collection of external peripherals. Device drivers provide the glue between the uEZ system, application, RTOS, and low-level HAL drivers. Therefore, the primary role of the platform is the definition of all available external hardware and how they connect.

7.2 Platform Files

All files for the platform are stored in the following directory path.

`/Source/Platform/<manufacturer>/<platform name>`

The platform file is usually broken into two files: BSP and Platform in the following format:

`/Source/Platform/<manufacturer>/<platform name>/<platform>_uEZBSP.c`
`/Source/Platform/<manufacturer>/<platform name>/<platform>_uEZPlatform.c`

The `<platform>_uEZBSP.c` file contains the following functions (if enabled in Config.h):

`UEZBSP_CPU_PinConfigInit()`
`UEZBSPSDRAMInit()`
`UEZBSPSerialInit()`
`UEZBSPSRAMInit()`
`UEZBSPHeapInit()`

The `<platform>_uEZPlatform.c` file contains the following required function:

`uEZPlatformInit()`

Unlike the processor directory where all HAL drivers are stored in this path, the platform stores all device drivers in the `/Source/Device` subdirectories.

7.3 Platform Configuration

All platform types are defined in the file `Include/Config.h`. Add the new platform to the list of platform types in `MANUFACTURER_PLATFORM` format and in all caps (e.g., `FDI_CARRIER`) and use the next sequential number. Change the `#define` `UEZ_PLATFORM` to be the newly defined `MANUFACTURER_PLATFORM` name.

To ensure that the correct files compile, in each platform .c file include the following header and footer:

Header:

`#include "Config.h"`
`#if (UEZ_PROCESSOR==MANUFACTURER_PLATFORM)`

Footer:

`#endif // (UEZ_PROCESSOR==MANUFACTURER_PLATFORM)`

7.4 Platform Initialization

In file >/<platform>_uEZPlatform.c, uEZPlatformInit () and is called in the boot sequence to register and configure all the hardware devices specific to this platform. At this point, the processor has already registered all of its devices and can be found using HALInterfaceFind. Most, if not all, of the device drivers will link to HAL devices.

At this point, the RTOS has been initialized, but is not running. The code must not use RTOS features other than to create tasks, semaphores, queues, etc. NOTE: Tasks will run when the RTOS is started.

7.4.1. Device Driver Registration and Initialization

Each device peripheral is initialized using the following steps.

7.4.1.1. Step 1: Register the Device Driver Interface

Call UEZDeviceTableRegister () with the name and device interface. If the hardware device is needed by another device, the return pointer to the workspace can be optionally returned. For example:

```
UEZDeviceTableRegister(  
    "I2C0",  
    (T_uezDeviceInterface *) &I2CBus_Generic_Interface,  
    0,  
    &p_i2c0);
```

In this case, the device driver is defined by I2CBus_Generic_Interface, will be defined in the device registry as "I2C0", and returns a pointer to the T_uezDeviceWorkspace in p_i2c0.

The device driver interfaces are defined inside each of the individual device drivers. For each interface, add the following line at the top:

```
extern const DEVICE_I2C_BUS I2CBus_Generic_Interface;
```

In this case, a DEVICE_I2C_BUS class type is defined for the interface. Each device will have a different interface type.

7.4.1.2. Step 2: Configure the Device Driver

Next call an appropriate configure routine if necessary. Each device driver will have a different routine for configuring, although some will have none. Device drivers usually depend on HAL drivers as well as each other. The returned workspace pointer can be passed into other configure routines linking together the drivers.

When configuration is required, a specific routine defined by that driver (with the word Configure in it) is used. To continue the I2C Bus initialization example, the following is used.

```
I2C_Generic_Configure(p_i2c0, "I2C0")
```

In this case, the I2C Bus driver wants the HAL driver registry name and will connect to it.

7.4.2. Additional Startup Steps

Additional functions can be called to start any libraries or tasks that are needed for running the platform. Again, at this point the RTOS is not running, so tasks can only be created but not executed. For example, the TCP/IP stack can configure and prepare its memory and root task, but will not be running.

7.4.3. Platform Startup Task

Once `uEZPlatformInit()` is completed, the task `uEZPlatformStartup` is created and the RTOS is started. `uEZPlatformStartup` is the second phase of initialization where the platform chooses to do any RTOS dependent initialization steps. It should finally end by creating the task called “MainTask” and starting `main()`. For example,

```
UEZTaskCreate(  
    (T_uezTaskFunction)MainTask,  
    "Main",  
    MAIN_TASK_STACK_SIZE,  
    0,  
    UEZ_PRIORITY_NORMAL,  
    &G_mainTask);
```

8.0 Appendix: Example Widget Driver Implementation

This section details how to create a new set of drivers to interface to a new hardware device called a Widget. For this example, let's assume that the Widget is a Widgetness device to turns on or off a light. Widgetness devices have two commands: TurnOn and TurnOff.

Most implementations have three layers: Library, Device Driver, and HAL Driver. In this example, we'll work from the bottom up.

8.1 HAL Driver

At the bottom most level, the HAL driver provides direct access to the hardware and assumes that it has exclusive control of the hardware on each function call. Because there may be multiple types of Widgets, each HAL Driver uses a HAL Driver Type structure which is defined in the Include/HAL directory. For this example, we'll start by making the file Include/HAL/Widget.h as follows:

```
// File: Include/HAL/Widget.h

#ifndef _WIDGET_H_
#define _WIDGET_H_
#include <uEZTypes.h>
#include <HAL/HAL.h>

typedef struct {
    T_halInterface iInterface;

    T_uezError (*TurnOn)(void *aWorkspace);
    T_uezError (*TurnOff)(void *aWorkspace);
} HAL_Widget;

#endif // _WIDGET_H_
```

The above file defines HAL_Widget. All Widget devices must use this interface style for future compatibility. It declares that widgets can be turned on or off and nothing else. The T_halInterface iInterface structure is a standard header on all HAL interfaces and will be explained below.

The HAL driver itself is defined in the /Source/Devices/{DeviceType}/{Company}/{SpecificDevice} directory where {DeviceType} is Widget in this case, {Company} is the maker of the device, and {SpecificDevice} is the particular model of widget. Assuming the company is Acme and we're working on the W1000, it would go to the directory "/Source/Devices/Widget/Acme/W1000".

File names will start with the model number followed usually by "main". In this case, we create the file "widget1000_main.c". Routines will also use a

“{DeviceType}_{Company}_{SpecificDevice}_{function}” name to avoid name conflicts and to aide debugging. For example, our widget HAL driver may appear as follows:

```
// File: widget1000_main.c

// Workspace definition
typedef struct {
    const HAL_Widget *iDevice;
    // private data goes here
} T_Widget_Acme_W1000_Workspace;

// Functions:
T_uezError Widget_Acme_W1000_TurnOn(void *aWorkspace)
{
    T_Widget_Acme_W1000_Workspace *p =
(T_Widget_Acme_W1000_Workspace *)aWorkspace;
    T_uezError error = UEZ_ERROR_NONE;

    // Driver specific code goes here using variable p
    return error;
}

T_uezError Widget_Acme_W1000_TurnOff(void *aWorkspace)
{
    T_Widget_Acme_W1000_Workspace *p =
(T_Widget_Acme_W1000_Workspace *)aWorkspace;
    T_uezError error = UEZ_ERROR_NONE;

    // Driver specific code goes here using variable p
    return error;
}

T_uezError Widget_Acme_W1000_InitializeWorkspace(void
*aWorkspace)
{
    T_Widget_Acme_W1000_Workspace *p =
(T_Widget_Acme_W1000_Workspace *)aWorkspace;
    T_uezError error = UEZ_ERROR_NONE;

    // Set initial values in workspace
    return error;
}

const HAL_Widget Widget_Acme_W1000_Widget0 = {
    "Widget:Acme:W1000:Widget0",
    0x0100,
    Widget_Acme_W1000_InitializeWorkspace,
    sizeof(T_Widget_Acme_W1000_Workspace),
};
```

There are four main sections to the HAL driver. First, there is the workspace. This represents a private workspace for this instance of the driver. For example, there may be 4 widgets on this hardware platform that use the same HAL interface. Each one has its own workspace. Second, there are the functions that operate on the workspace. Each function represents one unit of work that must be completed before another function (typically) is called. Third, is the routine xxx_InitializeWorkspace which is called once to setup the workspace. Finally, the HAL interface is defined and links up the above 3 sections. It provides a unique name for the interface, its version number (for compatibility tracking), the initialization routine, and the size of the workspace required.

When the BSP starts, it will register all HAL driver interfaces to the system and instantiate all the workspaces. It will do this by calling HALInterfaceRegister and pass in the interface structure. In this case, it is Widget_Acme_W1000_Widget0.

After registration, other software can find it by calling HALInterfaceFind and getting a pointer to the workspace. All workspaces start with a pointer to the interface to provide easy access to pointers to the interface routines. For example, if you have a T_halWorkspace * pointer to a Widget0 device, you would use the following syntax to turn the Widget on:

```
((T_Widget_Acme_W1000_Workspace *)p)->TurnOn(p);
```

Widget0's TurnOn function is called and it is given the private workspace.

For this example, let's assume that Widgets are only turned on for short periods of time and multiple tasks may be running and accessing the same Widget. In this case, the RTOS will need to semaphore the accesses. Since the HAL layer is built without knowing about a RTOS, a device driver is needed.

8.2 Device Driver

Device Drivers follow almost exactly the same rules as the HAL drivers. Each has its own workspace, interface, and registration. The main difference between the device driver and the HAL driver is how it interfaces with the RTOS.

For our widget example, we now define a system of 'widgets' that apply to all widgets that use the RTOS to control task blocking. Again, a new device driver type will be created, but this time it will be created in "Include/Device/Widget.h", as shown below:

```
// File: Include/Device/Widget.h
#include <uEZTypes.h>
#include <uEZDevice.h>

typedef struct {
    T_uezDeviceInterface iDevice;

    T_uezError (*Configure)(void *aWorkspace, HAL_Widget**
                           aWidget);
```

```

    T_uezError (*TurnOn)(void *aWorkspace);
    T_uezError (*TurnOff)(void *aWorkspace);
    T_uezError (*WaitTilOff)(void *aWorkspace);
} DEVICE_Widget;

```

Notice that we have a similar structure as HAL_Widget, but we've added WaitTilOff() and Configure() to the list. Configure() is used to link the Device Driver to a specific HAL Widget driver and workspace. WaitTilOff() demonstrates how more functionality can be added to at this level while being compatible with all hardware implementations.

The device driver may look like this:

```

// File: Source/Devices/Generic/Widget
#include <uEZ.h>
#include <Device/Widget.h>
#include <HAL/Widget.h>

typedef struct {
    const DEVICE_Widget iDevice;
    HAL_Widget **iWidget;
    T_uezSemaphore iSem;
} T_Widget_Generic_Workspace;

T_uezError Widget_Generic_InitializeWorkspace(void *aWorkspace)
{
    T_uezError error;
    T_Widget_Generic_Workspace *p = aWorkspace;

    // Setup semaphore for accesses
    error = UEZSemaphoreCreateBinary(&p->iSem);

    return error;
}

T_uezError Widget_Generic_TurnOn(void *aWorkspace)
{
    T_uezError error = UEZ_ERROR_NONE;
    T_Widget_Generic_Workspace *p = aWorkspace;

    // Attempt to grab the Widget's semaphore, or get a timeout
    immediately
    error = UEZSemaphoreGrab(p->iSem, 0);

    // Have the hardware turn on
    (*p->iWidget)->TurnOn(p->iWidget);

    return error;
}

T_uezError Widget_Generic_TurnOff(void *aWorkspace)
{
    T_uezError error = UEZ_ERROR_NONE;

```

```

    T_Widget_Generic_Workspace *p = aWorkspace;
    DEVICE_Widget *pw = (DEVICE_Widget *) (p->iDevice);

    // Attempt to release the Widget's semaphore, or timeout
    immediately
    error = UEZSemaphoreRelease(p->iSem, 0);

    // Turn off the unit
    (*p->iWidget)->TurnOff(p->iWidget);

    return error;
}

T_uezError Widget_Generic_WaitTilOff (void *aWorkspace)
{
    T_uezError error = UEZ_ERROR_NONE;
    T_Widget_Generic_Workspace *p = aWorkspace;

    // Attempt to grab the Widget's semaphore, implying it is now
    off and able
    // to turn off.
    error = UEZSemaphoreGrab(p->iSem, UEZ_TIMEOUT_INFINITE);
    if (error)
        return error;

    // Release semaphore, we had it when it was off
    error = UEZSemaphoreRelease(p->iSem, 0);

    return error;
}

T_uezError Widget_Generic_Configure(void *aWorkspace, HAL_Widget
**aWidget)
{
    T_uezError error = UEZ_ERROR_NONE;
    T_Widget_Generic_Workspace *p = aWorkspace;

    p->iWidget = aWidget;

    return error;
}

const DEVICE_Widget Widget_Generic_Interface = {
    "Widget:Generic",
    0x0100,
    Widget_Generic_InitializeWorkspace,
    sizeof(T_Widget_Generic_Workspace),

    // Functions
    Widget_Generic_Configure,
    Widget_Generic_TurnOn,
    Widget_Generic_TurnOff,
    Widget_Generic_WaitTilOff

```

```
};
```

Like the HAL implementation, there are four main sections: the workspace definition, the workspace initialization, the routines that work on the workspace (and make calls to the RTOS and hardware), and the interface definition.

The `iWidget` field and `Configure()` routine needs to be explained. The `iWidget` field is a pointer to a pointer to the `HAL_Widget` interface. The first level pointer points to the workspace. Since all workspaces start with a pointer to their associated interfaces, this effectively is the same as a pointer to a pointer. By doing `iWidget`, the address is the workspace. By doing `*iWidget`, the resulting address is the workspace. In this way, a single pointer can be used to represent both two structures. When the following line is executed:

```
(*p->iWidget)->TurnOff(p->iWidget);
```

It means that the routine `TurnOff` is being called in `*iWidget` (the HAL's interface routines) and `iWidget` (the HAL Widget's workspace) is being passed to it.

The `Configure()` routine is called to link the device driver with a HAL driver. It can also be used to setup any other values in the private workspace as needed (e.g., default on/off state).

Finally, the platform BSP must call `choose` which devices it is using and register them in the device table with `UEZDeviceTableRegister()`. The interface is registered with a name and a unique handle is assigned to this instance. Device handles can be found by name using `UEZDeviceTableFind`. `UEZDeviceTableGetWorkspace` can be used to turn handles into workspaces. The BSP will also typically give the device a generic name so that the library and application layers can use the device without knowing any specific details about the hardware. For example, "SPI0" may be used for the first SPI bus in the system.

8.3 Library

The next layer is the library layer. Although applications can directly access the device drivers or HAL drivers, most users expect a simple-to-use interface. The library can serve as a simple wrapper around the device driver, or it can link the devices to other devices as needed. Below is an example widget library implementation:

```
// File: Library/ueZWidget.c
#include <Device/Widget.h>
T_uezError UEZWidgetOpen(char *aName, T_uezDevice *aDeviceHandle)
{
    T_uezError error;
    error = UEZDeviceTableFind(aName, aDeviceHandle);
    if (error)
        return error;
    // Turn widget on by default
```

```

        WidgetOn(*aDeviceHandle);
    }

T_uezError UEZWidgetOn(T_uezDevice aWidget)
{
    T_uezError error;
    DEVICE_Widget **p;

    // Get workspace or return error
    error = UEZDeviceTableGetWorkspace(aWidget,
(T_uezDeviceWorkspace *) &p);
    if (error)
        return error;

    return (*p)->TurnOn((void *)p);
}

T_uezError UEZWidgetOff(T_uezDevice aWidget)
{
    T_uezError error;
    DEVICE_Widget **p;

    // Get workspace or return error
    error = UEZDeviceTableGetWorkspace(aWidget,
(T_uezDeviceWorkspace *) &p);
    if (error)
        return error;

    return (*p)->TurnOn((void *)p);
}

T_uezError UEZWidgetWaitTilOff(T_uezDevice aWidget)
{
    T_uezError error;
    DEVICE_Widget **p;

    // Get workspace or return error
    error = UEZDeviceTableGetWorkspace(aWidget,
(T_uezDeviceWorkspace *) &p);
    if (error)
        return error;

    return (*p)->WaitTilOff((void *)p);
}

T_uezError UEZWidgetClose(T_uezDevice aWidget)
{
    // Turn widget off by default
    return WidgetOff(aDeviceHandle);
}

```

8.4 Summary

In conclusion, the above examples show a full bottom to top view of a complete device system. Note that most devices will use pieces of existing systems. For example, if a new widget type is developed, either a new device driver or HAL driver can be built without changing the other two components. The top-level application will not need to know about the specific hardware details. If the library is changed to improve the application experience, the hardware drivers will not need to be changed. Each level abstracts and protects the other layers from change.